Deep Q-Learning with Keras and Gym

In [1]:
```
import random
import gymnasium as gym
import numpy as np
from collections import deque
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

from tensorflow.keras import backend as K
import tensorflow as tf
```

DQN

In [2]:
```
class DQNAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.memory = deque(maxlen=2000)
        self.gamma = 0.95    # discount rate
        self.epsilon = 1.0  # exploration rate
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.model = self._build_model()

    def _build_model(self):
        # Neural Net for Deep-Q learning Model
        model = Sequential()
        model.add(Dense(24, input_dim=self.state_size, activation='relu'))
        model.add(Dense(24, activation='relu'))
        model.add(Dense(self.action_size, activation='linear'))
        model.compile(loss='mse',
                      optimizer=Adam(learning_rate=self.learning_rate))
        return model

    def memorize(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        if np.random.rand() <= self.epsilon:
            return random.randrange(self.action_size)
        act_values = self.model.predict(state, verbose=VERBOSE)
        return np.argmax(act_values[0])  # returns action

    def replay(self, batch_size):
        minibatch = random.sample(self.memory, batch_size)
        for state, action, reward, next_state, done in minibatch:
            target = reward
            if not done:
                # The essential Q-learning update...
                target = (reward + self.gamma *
                          np.amax(self.model.predict(next_state, verbose=VERBOSE
```

```
            target_f = self.model.predict(state, verbose=VERBOSE)
            target_f[0][action] = target

            # in this case we do a one-by-one update
            self.model.fit(state, target_f, epochs=1, verbose=VERBOSE)

        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay

    def load(self, name):
        self.model.load_weights(name)

    def save(self, name):
        self.model.save_weights(name)
```

DQN Batch

In [3]:
```
class DQNBatchAgent(DQNAgent):
    def replay(self, batch_size):
        minibatch = random.sample(self.memory, batch_size)
        states, targets_f = [], []
        for state, action, reward, next_state, done in minibatch:
            target = reward
            if not done:
                # The essential Q-learning update...
                target = (reward + self.gamma *
                          np.amax(self.model.predict(next_state, verbose=VERBOSE

            target_f = self.model.predict(state, verbose=VERBOSE)
            target_f[0][action] = target

            #---------------------------------------------
            # Filtering out states and targets for training
            states.append(state[0])
            targets_f.append(target_f[0])
            #---------------------------------------------

        # in this case we do a batch update
        history = self.model.fit(np.array(states), np.array(targets_f), epochs=1

        # Keeping track of loss
        loss = history.history['loss'][0]

        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
        return loss
```

DDQN

In [4]:
```
class DDQNAgent(DQNAgent):
    def __init__(self, state_size, action_size):
        super().__init__(state_size, action_size)
        # We additionally create a second "target_model"...
        self.target_model = self._build_model()
        self.update_target_model()
```

```
    def _build_model(self):
        # Neural Net for Deep-Q learning Model
        model = Sequential()
        model.add(Dense(24, input_dim=self.state_size, activation='relu'))
        model.add(Dense(24, activation='relu'))
        model.add(Dense(self.action_size, activation='linear'))
        model.compile(loss=self._huber_loss, # <-- this is the only difference w
                      optimizer=Adam(learning_rate=self.learning_rate))
        return model

    """Huber loss for Q Learning

    References: https://en.wikipedia.org/wiki/Huber_loss
                https://www.tensorflow.org/api_docs/python/tf/keras/losses/Huber
    """
    def _huber_loss(self, y_true, y_pred, clip_delta=1.0):
        error = y_true - y_pred
        cond  = K.abs(error) <= clip_delta

        squared_loss = 0.5 * K.square(error)
        quadratic_loss = 0.5 * K.square(clip_delta) + clip_delta * (K.abs(error)

        return K.mean(tf.where(cond, squared_loss, quadratic_loss))

    def update_target_model(self):
        # copy weights from model to target_model
        self.target_model.set_weights(self.model.get_weights())

    def replay(self, batch_size):
        minibatch = random.sample(self.memory, batch_size)
        for state, action, reward, next_state, done in minibatch:
            target = self.model.predict(state, verbose=VERBOSE)
            if done:
                target[0][action] = reward
            else:
                # a = self.model.predict(next_state)[0]
                t = self.target_model.predict(next_state, verbose=VERBOSE)[0]
                # The essential Q-learning update... but "t" comes from the "tar
                target[0][action] = reward + self.gamma * np.amax(t)
                # target[0][action] = reward + self.gamma * t[np.argmax(a)]

            # in this case we do a one-by-one update
            self.model.fit(state, target, epochs=1, verbose=VERBOSE)

        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
```

Agents Training

In [5]:
```
env = gym.make('CartPole-v1')
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

MODEL_TYPE = 'DQN' # 'DDQN' 'DQNBatch'
MODEL_TYPE = 'DQNBatch'
MODEL_TYPE = 'DDQN'
```

```
if MODEL_TYPE == 'DQN':        agent = DQNAgent(state_size, action_size)
elif MODEL_TYPE == 'DDQN':     agent = DDQNAgent(state_size, action_size)
elif MODEL_TYPE == 'DQNBatch': agent = DQNBatchAgent(state_size, action_size)

# agent.load(f"saved_models/cartpole-{MODEL_TYPE}.h5")

done = False
batch_size = 32
VERBOSE = 0

EPISODES = 5 # 5_000

for e in range(EPISODES):
    state,info = env.reset()
    state = np.reshape(state, [1, state_size])
    for time in range(500):
        # env.render()
        action = agent.act(state)

        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        if MODEL_TYPE in ['DQN','DQNBatch']:
            reward = reward if not done else -10
        elif MODEL_TYPE == 'DDQN':
            x,x_dot,theta,theta_dot = next_state
            r1 = (env.x_threshold - abs(x)) / env.x_threshold - 0.8
            r2 = (env.theta_threshold_radians - abs(theta)) / env.theta_threshol
            reward = r1 + r2

        next_state = np.reshape(next_state, [1, state_size])
        agent.memorize(state, action, reward, next_state, done)
        state = next_state

        if done:
            if MODEL_TYPE == 'DDQN':
                agent.update_target_model() # other "cyles" for this update coul

            print("episode: {}/{}, score: {}, e: {:.2}"
                  .format(e, EPISODES, time, agent.epsilon))
            break

        if len(agent.memory) > batch_size:
            loss = agent.replay(batch_size)
            if MODEL_TYPE == 'DQNBatch':
                # Logging training loss every 10 timesteps
                if time % 10 == 0:
                    print("-- episode: {}/{}, time: {}, loss: {:.4f}"
                        .format(e, EPISODES, time, loss))

    # if e % 10 == 0:
    #     agent.save(f"saved_models/cartpole-{MODEL_TYPE}.h5")
```

```
episode: 0/5, score: 22, e: 1.0
episode: 1/5, score: 62, e: 0.77
episode: 2/5, score: 21, e: 0.69
episode: 3/5, score: 15, e: 0.64
episode: 4/5, score: 165, e: 0.28
```